Online Training Program
- ASP.NET Core Web API Online Training

ASP.NET WEB API – Basics
- Introduction to ASP.NET Web API
- Creating ASP.NET Web API Application
- How to add Swagger in Web API
- How to use Fiddler to test Web API
- How to use POSTMAN to test Web API
- ASP.NET Web API using SQL Server
- Content Negotiation in Web API
- Media Type Formatter in Web API
- How to Implement GET Method in Web API
- How to Implement POST Method in Web API
- How to Implement PUT Method in Web API
- How to Implement DELETE Method in Web API
- Custom Method Names in Web API
- Parameter Binding in Web API
- Consuming Web API Service From jQuery
- Calling Web API Service in a Cross Domain Using jQuery AJAX
- Cross-Origin Resource Sharing in WEB API

WEB API Routing and Attribute Routing
- ASP.NET Web API Routing
- Routing Variations in WEB API
- Routing and Action Selection in Web API
- ASP.NET Web API Attribute Routing
- Optional Parameters in Web API Attribute Routing
- Route Prefix in Web API
- Web API Attribute Routing Route Constraints
- Route Names and Route Orders in Attribute Routing

ASP.NET WEB API – Security
- Authentication and Authorization in ASP.NET Web API
- ASP.NET Web API Basic Authentication
- Rate-Based Basic Authentication in Web API
- Web API Service with Basic Authentication
- **HTTP Message Handlers in WEB API**
- Basic Authentication Using Message Handler in Web API
- HTTP Client Message Handler in Web API
- Token Based Authentication in ASP.NET Web API
- Client Validation Using Basic Authentication in Web API
- Refresh Token in Web API
- Consume Refresh Token in C# Client
- HMAC Authentication in Web API

Web API Versioning
- Web API Versioning Using URI
- Web API Versioning using Query string Parameter
- Web API Versioning Using Custom Header
- Web API Versioning Using Accept Header
- Web API Versioning Using Custom Media Types

Advanced Concepts
- Enable SSL in Visual Studio Development Server
- Enable HTTPS in Web API

Popular ASP.NET Web API Books
- Most Recommended ASP.NET Web API Books

HTTP Message Handlers in WEB API

Back to: ASP.NET Web API Tutorials For Beginners and Professionals

Server-Side HTTP Message Handlers in ASP.NET Web API

In this article, I am going to discuss the **Server-Side HTTP Message Handlers in ASP.NET Web API** with Examples. Please read our previous article where we discussed **How to Consume Web API Services** with Basic Authentication. As part of this article, we are going to discuss the following important concepts related to Message handlers.

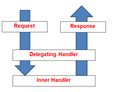
1. **What is an HTTP Message handler in ASP.NET Web API Application?**
2. **Understanding Delegating Handler in Web API:**
3. **Different Types of HTTP Message Handlers in Web API:**
4. **Understanding Server-Side HTTP Message Handlers in Web API:**
5. **How to Create Custom Server-Side HTTP Message Handlers in Web API?**
6. **How to add Custom HTTP Message Handlers to the Pipeline.**
7. **Different Types of Examples to understand the HTTP Message Handlers.**

What is an HTTP Message handler in ASP.NET Web API Application?

An **HTTP Message Handler in ASP.NET Web API** is a class that receives an HTTP request and returns an HTTP response. The **Message Handler** is derived from the abstract **HttpMessageHandler** class.

Understanding Delegating Handler in Web API:

To handle the HTTP Request and to generate the HTTP Response in ASP.NET Web API, a series of message handlers are chained together. The first handler in the chain receives the HTTP request. Do some processing, and gives the request to the next handler. At some point, the response is generated and goes back up in the chain. This pattern is called delegating handler. The following diagram shows the process.



What are the Different Types of HTTP Message Handlers available in ASP.NET Web API?

In ASP.NET Web API Framework, there are two types of message handlers are available. They are as follows

1. **Server-Side HTTP Message Handlers :**
2. **Client-Side HTTP Message Handlers**

In this article, I am going to discuss the **Server-Side HTTP Message Handlers** in the Web API Framework with real-time examples. In our next article, I am going to discuss the **Client-Side HTTP Message Handlers** in ASP.NET Web API.

Understanding Server-Side HTTP Message Handlers in Web API:

On the server-side, the ASP.NET Web API Framework uses some built-in message handlers which are as follows:

1. **HttpServer**: This built-in message handler gets the request from the host.
2. **HttpRoutingDispatcher**: This message handler dispatches the request based on the route.
3. **HttpControllerDispatcher**: This message handler sends the request to an ASP.NET Web API controller.

You can also create your own custom handlers and then add them to the Web API pipeline. The Message handlers are good for cross-cutting concerns (such as authentication and authorization) that operate at the level of HTTP messages rather than controller actions. For example, a custom message handler might do the following things:

1. **Read or modify the HTTP request headers.**
2. **Add a response header to the HTTP response.**
3. **Validate the requests before they reach the controller (i.e. Authentication and Authorization).**

The following diagram shows two custom handlers (Message Handler1 and Message Handler2) inserted into the Web API pipeline at the Server side.

Creating Custom Server-Side HTTP Message Handlers in Web API

As we already discussed along with the built-in Server-side Message handlers, you can also create your own Server-Side HTTP Message Handlers. So let's discuss how to create the **Custom Server-Side HTTP Message handlers** in ASP.NET Web API.

To create a custom Server-Side HTTP Message Handler in ASP.NET Web API, you need to create a class that must be derived from the **System.Net.Http.DelegatingHandler**. That custom class then should override the **SendAsync** method. **SendAsync** method has the following signature:

```
protected async override Task<HttpResponseMessage> SendAsync(
    HttpRequestMessage request, CancellationToken cancellationToken)
```

The **SendAsync** method takes an **HttpRequestMessage** as input and asynchronously returns an **HttpResponseMessage**. A typical implementation does the following:

1. Process the request message
2. Call the **base.SendAsync** method to send the request to the inner handler.
3. The inner handler returns a response message. (This step is asynchronous.)
4. Process the response message and returns the response to the caller.

A sample code is shown below.

```
using System.Linq;
using System.Net.Http;
using System.Threading;
using System.Threading.Tasks;

namespace MessageHandler.Models
{
    public class MessageHandler1 : DelegatingHandler
    {
        protected async override Task<HttpResponseMessage> SendAsync(
            HttpRequestMessage request, CancellationToken cancellationToken)
        {
            Debug.WriteLine("Process request");
            // Call the inner handler.
            var response = await base.SendAsync(request, cancellationToken);
            Debug.WriteLine("Process response");
            return response;
        }
    }
}
```

Note: The call to the **base.SendAsync** is asynchronous. If the handler does any work after this call, use the await keyword, as shown in the above example.

A delegating handler can also skip the inner handler and directly create the response. Let's look at the below example which exactly does the same thing.

```
using System.Net;
using System.Net.Http;
using System.Threading;
using System.Threading.Tasks;

namespace MessageHandler.Models
{
    public class MessageHandler2 : DelegatingHandler
    {
        protected override Task<HttpResponseMessage> SendAsync(
            HttpRequestMessage request, CancellationToken cancellationToken)
        {
            // Create the response.
            var response = new HttpResponseMessage(HttpStatusCode.OK)
            {
                Content = new StringContent("Hello!")
            };

            // Note: TaskCompletionSource creates a task that does not contain a
Delegate.
            var tcs = new TaskCompletionSource<HttpResponseMessage>();
            tcs.SetResult(response);
            return tcs.Task;
        }
    }
}
```

When a delegating handler creates the response without calling the **base.SendAsync** method, then the request skips the rest of the pipeline. This can be useful for a handler that validates the request creating an error response. We will discuss this with a complete example in our upcoming article.

How to Add Custom HTTP Message Handlers to the Pipeline in ASP.NET Web API?

To add the Custom HTTP Message Handlers on the server-side, you need to add the Custom HTTP Message Handlers to the **HttpConfiguration.MessageHandlers** collection inside the Register method of the **WebApiConfig** class as shown in the below image.

```
public static class WebApiConfig
{
    public static void Register(HttpConfiguration config)
    {
        config.MessageHandlers.Add(new MessageHandler1());
        config.MessageHandlers.Add(new MessageHandler2());

        // Web API routes
        config.MapHttpAttributeRoutes();

        config.Routes.MapHttpRoute(
            name: "DefaultApi",
            routeTemplate: "api/{controller}/{id}",
            defaults: new { id = RouteParameter.Optional }
        );
    }
}
```

Registering the Custom Message Handlers

The Message Handlers are going to call in the same order as they appear in the **MessageHandlers** collection. The reason is they are nested and the response message travels in the other direction. That is, the last handler in the first one to get the response message.

Notice that you don't need to set the inner handlers. That is the job of the Web API framework which will automatically connect the inner message handlers. Now let's look at some of the examples of custom message handlers.

Example: X-HTTP-Method-Override

The **X-HTTP-Method-Override** is a non-standard HTTP header. It is basically designed for clients who cannot send certain HTTP request types, such as PUT or DELETE. Instead, the client sends a POST request and sets the **X-HTTP-Method-Override** header to the desired method type. For example **X-HTTP-Method-Override: PUT**

So to support this, we need to create a custom message handler that adds support for X-HTTP-Method-Override.

```
using System.Linq;
using System.Net.Http;
using System.Threading;
using System.Threading.Tasks;

namespace MessageHandler.Models
{
    public class XHTTPMethodOverrideHandler : DelegatingHandler
    {
        readonly string[] _methods = { "DELETE", "HEAD", "PUT" };
        const string _header = "X-HTTP-Method-Override";

        protected override Task<HttpResponseMessage> SendAsync(
            HttpRequestMessage request, CancellationToken cancellationToken)
        {
            // Check for HTTP POST with the X-HTTP-Method-Override header.
            if (request.Method == HttpMethod.Post && request.Headers.Contains(_header))
            {
                // Check if the header value is in our methods list.
                var method = request.Headers.GetValues(_header).First(StrEqual);
                if (_methods.Contains(method,
StringComparer.InvariantCultureIgnoreCase))
                {
                    // Change the request method.
                    request.Method = new HttpMethod(method);
                }
            }
            return base.SendAsync(request, cancellationToken);
        }
    }
}
```

Explanation of the above code:

In the above **SendAsync** method, the handler checks whether the request message is a POST request and whether it contains the **X-HTTP-Method-Override** header. If so, then it validates the request header value and then modifies the request method. Finally, the handler calls the base.SendAsync to pass the message to the next handler.

When the request reaches the **HttpControllerDispatcher** class, **HttpControllerDispatcher** will route the request based on the updated request method.

Example: Adding a Custom Response Header

Let's see an example of a message handler that adds a custom header to every response message:

```
using System.Net;
using System.Net.Http;
using System.Threading;
using System.Threading.Tasks;

namespace MessageHandler.Models
{
    public class CustomHeaderHandler : DelegatingHandler
    {
        async protected override Task<HttpResponseMessage> SendAsync(
            HttpRequestMessage request, CancellationToken cancellationToken)
        {
            HttpRequestMessage response = await base.SendAsync(request,
cancellationToken);
            response.Headers.Add("X-Custom-Header", "This is my custom header.");
            return response;
        }
    }
}
```

In the above example, the handler first calls the **base.SendAsync** method to pass the request to the inner message handler. The inner handler returns a response message, but it does so asynchronously by using the **Task<T>** object. The response message is not available until the **base.SendAsync** completes asynchronously.

Example: Checking for an API Key

Some web services require the clients to include an API key in their request. The following example shows how a message handler can check the requests for a valid API key:

```
using System.Net;
using System.Net.Http;
using System.Threading;
using System.Threading.Tasks;

namespace MessageHandler.Models
{
    public class ApiKeyHandler : DelegatingHandler
    {
        public string Key { get; set; }

        public ApiKeyHandler(string key)
        {
            this.Key = key;
        }

        protected override Task<HttpResponseMessage> SendAsync(
            HttpRequestMessage request, CancellationToken cancellationToken)
        {
            if (ProcessApiKey(request))
            {
                var response = new HttpResponseMessage(HttpStatusCode.BadRequest);
                var tcs = new TaskCompletionSource<HttpResponseMessage>();
                tcs.SetResult(response);
                return tcs.Task;
            }
            return base.SendAsync(request, cancellationToken);
        }

        private bool ValidateKey(HttpRequestMessage message)
        {
            var query = message.RequestUri.ParseQueryString();
            string key = query["key"];
            return (key == Key);
        }
    }
}
```

The above message handler looks for the API key in the URI query string. (For the above example, we assume that the key is a static string. A real implementation would probably use more complex validation.) If the query string contains the key, the handler passes the request to the inner handler.

If the request does not have a valid key, the handler creates a response message with status 400, Forbidden. In this case, the handler does not call **base.SendAsync**, so the inner handler never receives the request, nor does the controller. Therefore, the controller can assume that all incoming requests have a valid API key.

Note: If the API key applies only to certain controller actions, consider using an action filter instead of a message handler. Action filters run after URI routing is performed.

In the next article, I am going to discuss **how to implement basic authentication using a message handler** in ASP.NET Web API. Here, in this article, I try to explain **Server-Side HTTP Message Handlers in WEB API** with real-time examples.

Dot Net Tutorials
About the Author: Pranaya Rout
Pranaya Rout has published more than 3,000 articles in his 11 year career. Pranaya Rout has very good experience with Microsoft Technologies, including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

1 thought on “HTTP Message Handlers in WEB API”

SUNNY
MARCH 6, 2020 AT 9:05 PM

Thank you, very clear. HttpMessageHandler acts similarly as middleware.

Reply

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name* | Email* | Website

Post Comment

© Dot Net Tutorials | Website Design by Sunnyo Pixel